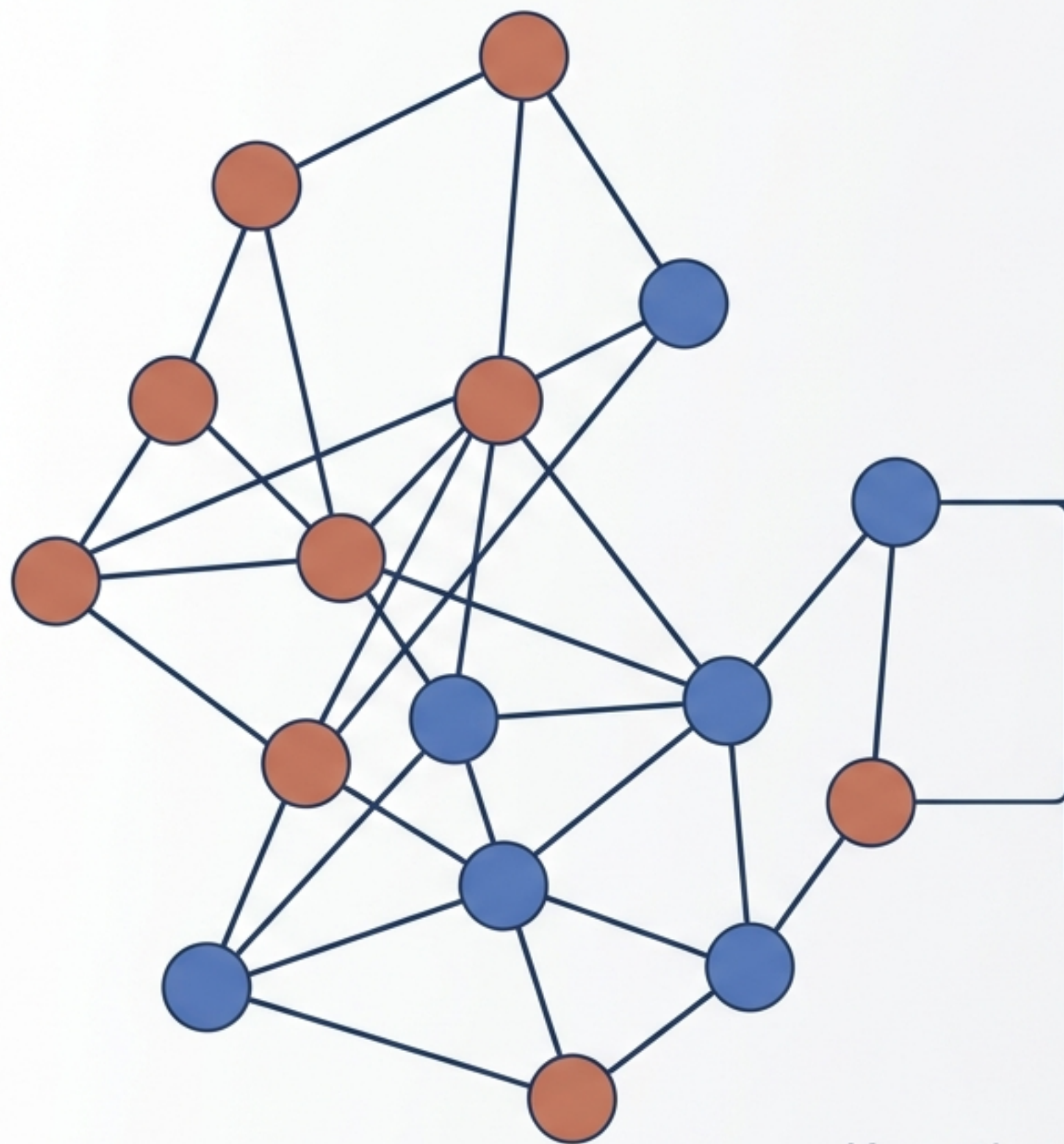




```
---
title: "The AI Memory Architecture"
date: 2024-05-22
author: "System Architect"
tags: [LLM, Knowledge Graph, RAG, Wiki]
version: 1.2.0
---
```

Introduction

This document outlines the transition from organic knowledge representation to structured documentation.

Core Concepts

- **Node Integration:** Mapping graph nodes to markdown sections.
- **Edge Transformation:** Converting relationships into explicit links and hierarchy.

Implementation Details

The system uses a hybrid approach, leveraging both 'cobalt-blue' automated indexing and 'terracotta' manual curation for optimal results.

```
def snap_to_grid(graph_data):
    # Transform organic graph to structured format
    return structured_document
```

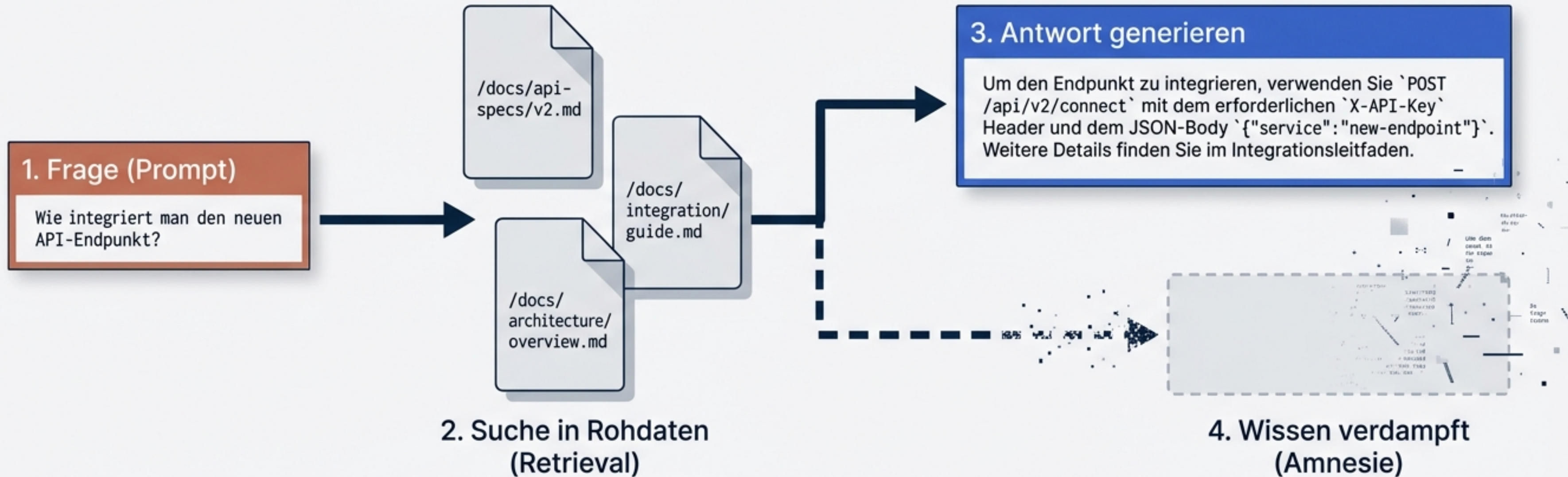
- File Path: /docs/architecture/ai_memory_v1.md
- Status: **Immutable Source**

Das Gedächtnis der KI

Karpathys LLM-Wiki, Google OKF und das Ende der RAG-Amnesie.

Das Problem mit RAG: Der „Murmeltier“-Effekt

Der „Murmeltier“-Effekt



RAG leitet Antworten bei jeder Frage von Grund auf neu ab. Es sucht, aber es lernt nicht. Es gibt keinen Zinseszins-Effekt (Compounding) für das Wissen.

Der Paradigmenwechsel: Vom Suchen zum Kompilieren

Obsidian ist die IDE,
das LLM ist der
Programmierer, das
Wiki ist die
Codebasis.

– Andrej Karpathy

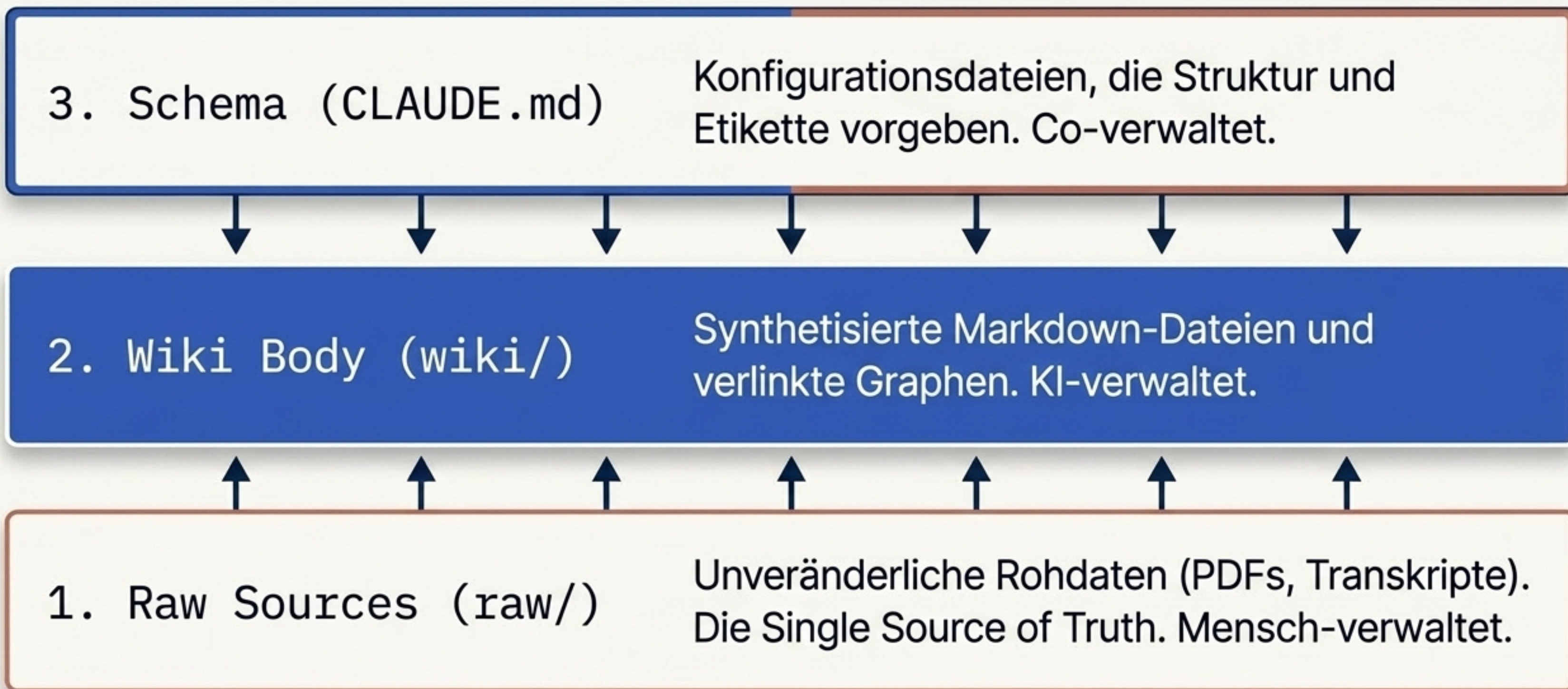
Menschliche Rolle

Quellen kuratieren. Richtung
vorgeben. Fragen stellen.

KI-Rolle

Wissen ablegen. Zusammenfassen.
Querverweise aktualisieren. (Bis zu
15 Dateien gleichzeitig bearbeiten).

Die 3 Schichten des LLM-Wikis



Der Zinsezins-Motor: Ingest, Query, Lint

Ingest, Query

Lint (Pflegen)
Regelmäßige
Gesundheitschecks
der Codebasis. Die KI
sucht nach
Widersprüchen,
verwaisten Seiten und
veralteten Fakten.



Ingest (Einspeisen)

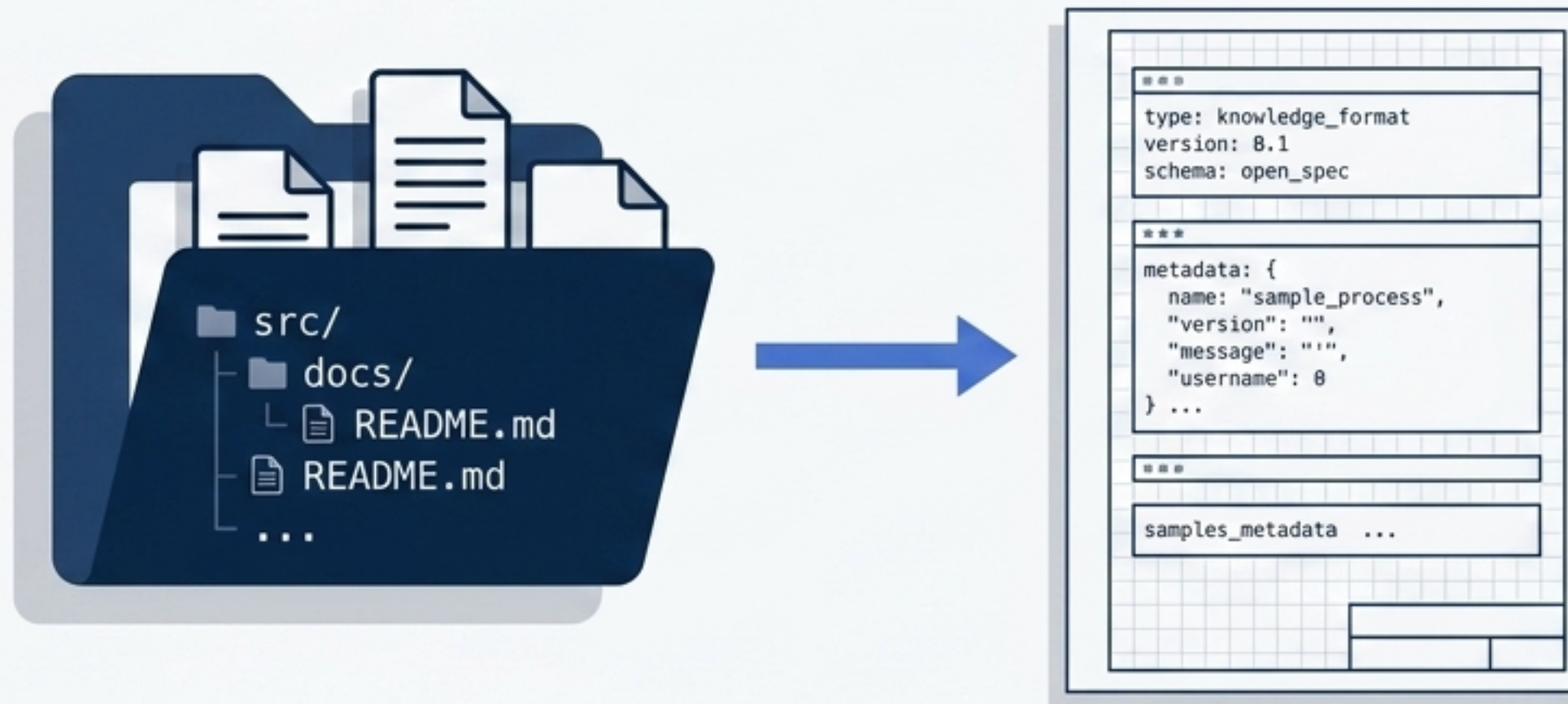
Die KI liest eine neue
Quelle, aktualisiert
`index.md` und passt 10-15
bestehende Dateien an.

Query (Abfragen)

Die KI navigiert über den
Index, synthetisiert
Antworten und
speichert brillante
Ergebnisse als dauerhaft
neue Wiki-Seiten.

Die Standardisierung: Google OKF (Open Knowledge Format)

Antworten mit einem Format, nicht mit einem Service.



- Kein SDK.
- Keine neue Datenbank.
- Kein Vendor Lock-in.

Entwickelt vom Google Cloud Data Team (v0.1).
Ein herstellerneutraler Standard für das Gedächtnis von KI-Agenten.

Die Anatomie eines OKF-Bundles

Dateibaum

```
okf-bundle/  
├── index.md  
└── tables/  
    └── orders.md — (Dateipfad =  
                    Concept ID)
```

orders.md

```
---  
type: BigQuery Table — (Das einzige Pflichtfeld!)  
title: Orders  
---
```

Orders Tabelle

Hier steht der Inhalt...

Die 3 OKF-Designprinzipien



Just Files

„Wer eine Datei per cat lesen kann, kann OKF lesen.“

Überall hostbar (Git, lokales Laufwerk).



Minimalistisch

Keine starren Taxonomien. Nur das Feld type ist verpflichtend, alles andere ist optional (Progressive Disclosure).



Extreme Toleranz

Kaputte Links (Broken Links) sind erlaubt – sie repräsentieren einfach Wissen, das noch nicht geschrieben wurde.

Systemvergleich: LLM-Wiki vs. RAG

Dimension	Klassisches RAG	LLM-Wiki (OKF)
Zustand	Stateless (rechnet bei jeder Frage neu)	Stateful (Wissen wird einmalig kompiliert)
Wissensaufbau	Ephemer (Antworten verdampfen)	Compounding (Wissen akkumuliert dauerhaft)
Sweet Spot	Unstrukturierter Long-Tail (Millionen Tokens)	Kuratiertes Kernwissen (< 100k Tokens)
Widersprüche	Werden oft übersehen	Werden durch Linting systematisch markiert

Die Grenzen des Formats (Ehrliche Einordnung)



Es skaliert sich nicht selbst.

OKF erfordert menschliche Kurationsarbeit und einen externen KI-Agenten. Ohne Pflege veraltet das Wissen.



Das Messy Librarian Problem.

Sprachmodelle machen Fehler. Sie können Links zu nicht-existenten Dateien erfinden oder Markdown-Strukturen zerstören.



Semantische Duplikation.

Wenn mehrere Agenten gleichzeitig schreiben, kann derselbe Fakt in verschiedenen Worten doppelt abgelegt werden.

Die Synthese: Der AI Context Stack

Das Gedächtnis (OKF / LLM-Wiki)

Liefert die kanonischen 80% der Wahrheit.
Dauerhaft, kuratiert, hochpräzise.

Die Reichweite (RAG)

Durchsucht die unstrukturierten 20% (Long-Tail, Millionen alte Support-Tickets).

Die Hände (MCP - Model Context Protocol)

Bietet Live-Zugriff auf aktuelle Datenbanken
und führt Aktionen aus.

Konkrete Anwendungsfälle für Unternehmen



Meeting Intelligence

Transkripte werden eingespeist. Die KI extrahiert Entscheidungen und aktualisiert automatisch Projektseiten und verlinkte Graphen.



Dynamisches Onboarding-Wiki

Statt statischer Confluence-Seiten wächst das System mit den Fragen neuer Mitarbeiter. Antworten werden zu neuen, dauerhaften Artikeln.



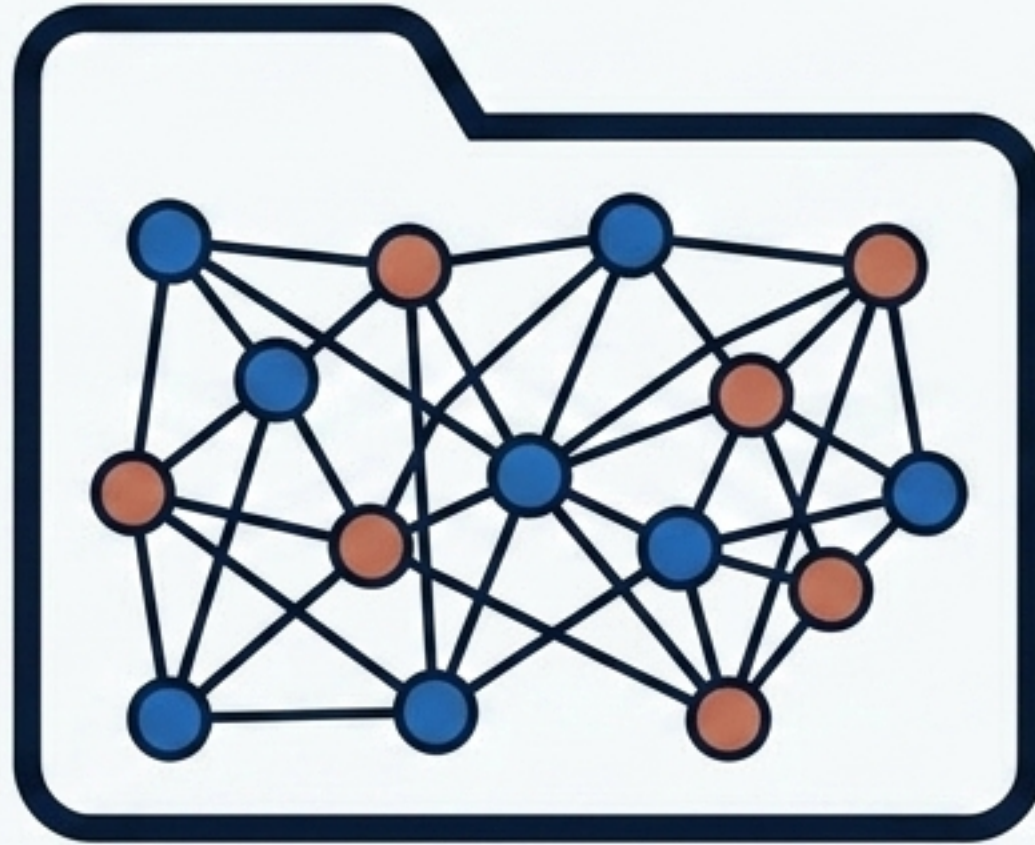
Semantische Datenkataloge

(Googles Referenz-Agent): Durchforstet BigQuery-Tabellen und erstellt eigenständig Concept-Pages für Metriken, Schemata und Join-Pfade.

Der pragmatische Start: Morgen umsetzen

- > 1. IDE wählen: Installiere Obsidian (als lokales Markdown-Frontend).
- > 2. Struktur anlegen: Erstelle die Ordner raw/ und wiki/.
- > 3. Regeln definieren: Schreibe eine CLAUDE.md (oder AGENTS.md), die dem LLM erklärt, wie dein Wiki organisiert ist.
- > 4. Ingest starten: Öffne Claude Code im Terminal und lass das LLM das erste Dokument verarbeiten und verlinken.

Fazit: Wissen, das wächst



RAG liefert Antworten für den Moment. Ein LLM-Wiki (standardisiert durch OKF) baut dauerhaftes, institutionelles Gedächtnis auf. Das LLM ist der Maintainer, das Wiki ist Ihre wertvollste Codebasis.